

Project 1: Real-World Dual-Core Sentry Application

This production project demonstrates an Acoustic & Spatial Sentinel Node configured entirely from scratch under the FreeRTOS framework using the ESP-IDF v6.0.2 Component Architecture.

Complete System Architecture Topology:

The software system splits tracking, serialization, and hardware control asynchronously across separate physical processing elements:

[CORE 0: Real-Time Sensing Hub]

- Polls digital PIR hardware lines at 50Hz (20ms intervals)
- Packages environment struct states into an atomic record
- Streams datasets over an 8-slot FreeRTOS Queue
- Drives high-frequency visual heartbeat task execution loops

|

▼ (FIFO Queue Conveyor Belt)

|

[CORE 1: Safety Logic & Actuation Engine]

- Suspends in an OS Blocked State until data drops into Queue
- Resolves boolean trigger equations instantly upon waking
- Requests ownership of Serial console interface via Mutex
- Outputs system analytics logs and switches alarm LED lines

Code:

```
#include <stdio.h>
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "freertos/queue.h"
#include "freertos/semphr.h"
#include "driver/gpio.h"

#define PIR_GPIO_PIN 4
#define MIC_GPIO_PIN 34
#define SECURITY_LED_PIN 2
#define STATUS_LED_PIN 18
```

```

// Structure of packet that will be passed through our queue

typedef struct{
    int motion_detected;
    int sound_detected;
} SentryData_t;

// Kernel handles
QueueHandle_t sentry_queue;
SemaphoreHandle_t console_mutex;

// Core 0: Producer - The sensing hub

void hardware_sensing_task(void *pvParameters){
    SentryData_t environment;

    while(1){
        // 1. Reading PIR sensor data ( 1 = Motion, 0 = Clear)
        environment.motion_detected = gpio_get_level(PIR_GPIO_PIN);
        // 2. Reading Microphone data
        environment.sound_detected = gpio_get_level(MIC_GPIO_PIN);
        // 3. Blinking the LED to visually prove Core 0 is sampling the sensor
        readings
        gpio_set_level(STATUS_LED_PIN, 1);
        vTaskDelay(20 / portTICK_PERIOD_MS);
        gpio_set_level(STATUS_LED_PIN, 0);
        // 4. Pushing the structured packet to the queue
        xQueueSend(sentry_queue, &environment, 0);
        // High frequency polling every 100 milliseconds
        vTaskDelay(100 / portTICK_PERIOD_MS);
    }
}

// Core 1: Consumer - Safety Logic and Actuation

void actuation_engine_task(void *pvParameters){
    SentryData_t report;

    while(1){
        // Blocks instantly until Core 0 uploads and execution packet
        if(xQueueReceive(sentry_queue, &report, portMAX_DELAY) == pdPASS){
            // Logic: Trip the security alert if sound or motion triggered
            if(report.motion_detected == 1 || report.sound_detected == 1){
                gpio_set_level(SEcurity_LED_PIN, 1);
            } else{
                gpio_set_level(SEcurity_LED_PIN, 0);
            }
        }
        // Mutex lock to print without serial line scrambling
    }
}

```

```

        if(xSemaphoreTake(console_mutex, portMAX_DELAY) == pdTRUE){
            printf("\n=====\\n");
            printf("[SENTRY CORE 1] Active Core ID: %d\\n",
xPortGetCoreID());
            printf("Motion Input Pin Status: %s\\n", report.motion_detected
? "🚨 INTRUSION!" : "Quiet");
            printf("Microphone Sound Spike: %s\\n",
report.sound_detected ? "🔊 NOISE CAPTURED!" : "Silent");
            printf("=====\\n");

            xSemaphoreGive(console_mutex);
        }
    }
}

// Component pin initialization
void init_hardware_pins(void){
    // Outputs
    gpio_reset_pin(SEcurity_LED_PIN);
    gpio_set_direction(SEcurity_LED_PIN, GPIO_MODE_OUTPUT);

    gpio_reset_pin(STATUS_LED_PIN);
    gpio_set_direction(STATUS_LED_PIN, GPIO_MODE_OUTPUT);

    // Inputs
    gpio_reset_pin(PIR_GPIO_PIN);
    gpio_set_direction(PIR_GPIO_PIN, GPIO_MODE_INPUT);
    gpio_set_pull_mode(PIR_GPIO_PIN, GPIO_PULLDOWN_ONLY);

    gpio_reset_pin(MIC_GPIO_PIN);
    gpio_set_direction(MIC_GPIO_PIN, GPIO_MODE_INPUT);
    gpio_set_pull_mode(MIC_GPIO_PIN, GPIO_PULLUP_ONLY);
}

// Main function
void app_main(void){
    printf("[BOOT] Launching Acoustic Sentry architecture .. \\n");
    init_hardware_pins();

    sentry_queue = xQueueCreate(8, sizeof(SentryData_t));
    console_mutex = xSemaphoreCreateMutex();

    if(sentry_queue != NULL && console_mutex != NULL){
        // Sensing core
        xTaskCreatePinnedToCore(hardware_sensing_task, "Sensing Hub", 3072,
NULL, 5, NULL, 0);
        // Logic and Actuation core

```

```

        xTaskCreatePinnedToCore(actuation_engine_task, "Actuator", 3072, NULL,
5, NULL, 1);
    }
}

```

Output:

```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  ESP-IDF

Microphone Sound Spike:  Silent
=====

[SENTRY CORE 1] Active Core ID: 1
Motion Input Pin Status: Quiet
Microphone Sound Spike:  Silent
=====

[SENTRY CORE 1] Active Core ID: 1
Motion Input Pin Status: Quiet
Microphone Sound Spike:  Silent
=====

[SENTRY CORE 1] Active Core ID: 1
Motion Input Pin Status: 🚨 INTRUSION!
Microphone Sound Spike:  Silent
=====

[SENTRY CORE 1] Active Core ID: 1
Motion Input Pin Status: 🚨 INTRUSION!
Microphone Sound Spike:  Silent
=====

[SENTRY CORE 1] Active Core ID: 1
Motion Input Pin Status: 🚨 INTRUSION!
Microphone Sound Spike:  Silent
=====

```

Build System Integration Requirement (main/CMakeLists.txt)

Under the modular architecture introduced in **ESP-IDF v6.x**, hardware drivers must be declared explicitly as standalone dependencies to map internal workspace header tracking trees accurately.

CMake

```
idf_component_register(SRCS "hello_world_main.c"
```

```

    INCLUDE_DIRS "."

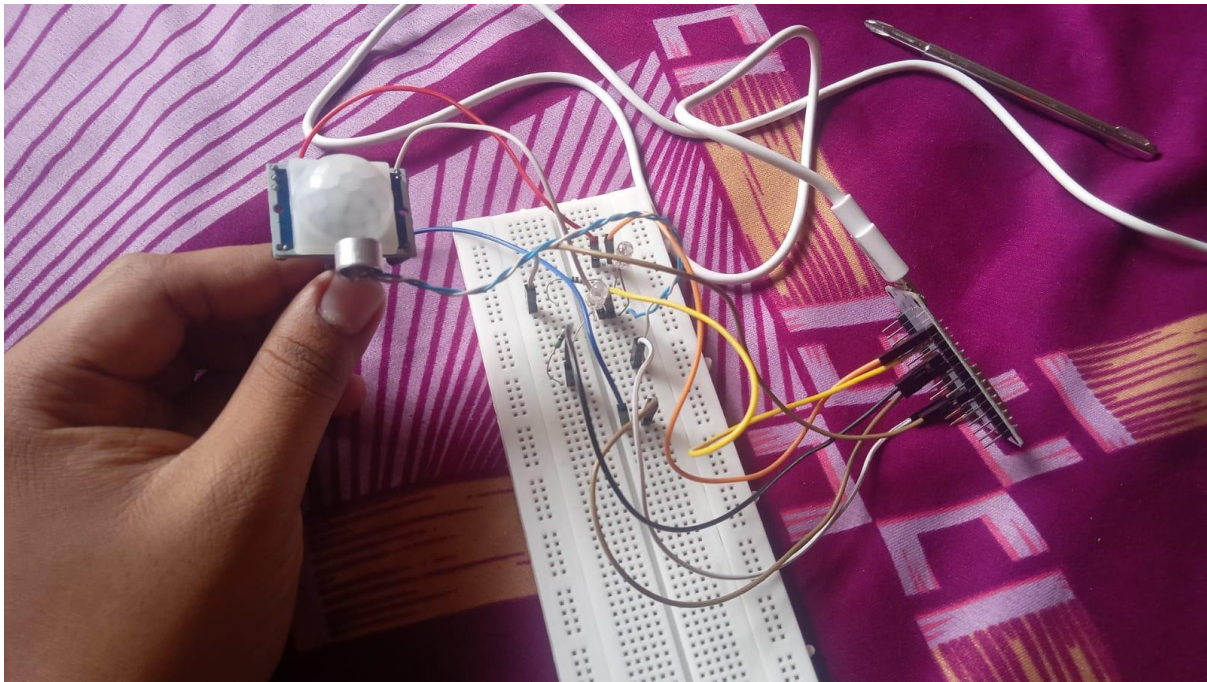
```

```

    REQUIRES esp_driver_gpio)

```

Implementation:



Key Insights and Verification Behaviors

- **Hardware Hardware Latency vs RTOS Performance:** The underlying execution code processes sensor statuses at 50 Hz, generating near-instantaneous queue scheduling steps. Any apparent delay under basic physical testing is caused exclusively by internal pyroelectric stabilizing curves native to raw PIR component sensor hardware.
- **True Concurrency Execution:** System outputs print Active Core ID: 1, confirming that tracking processes on Core 0 run completely independently from the consumer analysis loops on Core 1 without resource contention or temporal starvation.